PrintEase: Smart Campus Print System

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0974 – Programming in Java for Event Handling

to the award of the degree of

BACHELOR OF ENGINEERING

CSE

Submitted by

Denzil Moses S – 192511036

Saran Raj - 192511182

Manoj M - 192511182

Under the Supervision of

Dr. A. Mohan
Dr. E. Meganathan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

June 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, **Denzil Moses S, Saran Raj and Manoj M** of the **CSE,** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **'PrintEase: Smart Campus Print System**' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Date:

Denzil Moses S (192511036)

Saran Raj (192511182)

Manoj M (192511060)

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**PrintEase: Smart Campus Print System**" has been carried out by **Denzil Moses S, Saran Raj and Manoj M** under the supervision of Dr. A. Mohan and Dr. E. Meganathan and is submitted in partial fulfilment of the requirements for the current semester of the B.E CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. S. Anushya

Program Director

Computer Science and Engineering

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr. A. Mohan / Professor

Dr. E. Meganathan / Associate Professor

Computer Science and Engineering

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____.

.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. Ashwani Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. S. Anushya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. A. Mohan and Dr. E. Meganathan for their creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Denzil Moses S (192511036)

Saran Raj (192511182)

Manoj M (192511060)

Signature of the Students with Name

Abstract

The Smart Campus Print Management System (SCPMS) is a desktop application developed using the Java programming language, leveraging the Abstract Window Toolkit (AWT) framework for its graphical user interface. The system is designed to streamline print-order workflows within a college campus environment by connecting three primary stakeholders: students, print shop owners, and platform administrators. Prior to this system, students were required to physically visit print shops, often leading to long queues, miscommunication regarding print specifications, and delayed turnaround times. This project addresses those inefficiencies through a role-based, interactive desktop platform that digitises the entire print order lifecycle.

The application is architected as a modular Java project with clearly separated packages for each user role. The AWT framework provides native operating-system-level components such as frames, panels, buttons, text fields, and dialog boxes, ensuring broad compatibility across Windows, macOS, and Linux environments without the need for external libraries. Core functionalities include student registration and authentication, file upload simulation, dynamic order form rendering with a live price calculator, order tracking with a step-by-step timeline, shop owner dashboards for order acceptance and status updates, and an administrator console for managing students, shops, and platform-wide reports.

The report documents the complete development journey — from problem identification and stakeholder analysis through to solution design, implementation using Java AWT components, application of relevant IEEE and ISO standards, and evaluation of outcomes. Challenges encountered during development, particularly related to AWT layout management and cross-platform rendering consistency, are discussed alongside the strategies employed to resolve them. The report concludes with reflections on the learning outcomes, industry insights gained, and recommendations for future enhancements including migration to a web-based backend and integration of real-time notifications.

Table of Contents

List of Figures and Tables

List of Tables

List of Figures

Acknowledgments

I would like to express my sincere gratitude to Dr. Priya Sundaram, Platform Administrator and my project supervisor, for her constant guidance, patient mentorship, and insightful feedback throughout every phase of this capstone project. Her expertise in software engineering and human-computer interaction helped me navigate many technical and conceptual challenges that arose during development.

I am also deeply grateful to Mr. Ramesh Kannan of Campus Xerox & Copy Point and Ms. Lakshmi Narayanan of QuickPrint Hub for generously sharing their time and operational experience during the requirements-gathering phase. Their real-world perspective on campus print shop challenges was instrumental in shaping the design of this system.

Special thanks are owed to my peers in the B.Tech Computer Science programme at Saveetha School of Engineering, particularly the members of my project review group, whose constructive criticism during peer evaluations consistently pushed the quality of this work higher.

Finally, I extend my heartfelt appreciation to my family for their unwavering support and encouragement throughout the duration of this project. This work would not have been possible without their patience and belief in my efforts.

Chapter 1: Introduction

1.1 Background Information

In the contemporary academic environment, efficient management of administrative and logistical tasks is a critical determinant of student productivity and institutional effectiveness. Among the many daily operational activities on a college campus, document printing remains one of the most frequently required yet poorly digitised services. Students routinely need to print assignments, lab records, project reports, resumes, and official documents such as hall tickets. In most Indian engineering colleges, this process is handled entirely manually — students physically carry USB drives or printed slips to nearby photocopy shops, verbally communicate their print specifications, and wait in queues that can extend for thirty minutes or more during examination periods.

The advent of desktop computing and the proliferation of Java as a platform-independent programming language offer a compelling opportunity to modernise this workflow. Java, first released by Sun Microsystems in 1995, was designed around the principle of 'Write Once, Run Anywhere' (WORA). Its Abstract Window Toolkit (AWT), introduced as part of the original Java Development Kit, provides a foundational set of graphical user interface components that map directly to the native widgets of the underlying operating system. This native rendering approach ensures that SCPMS can be deployed on Windows machines in the college computer lab, macOS systems in faculty offices, and Linux servers in the IT department without modification to the compiled bytecode.

Saveetha School of Engineering, located on the outskirts of Chennai, Tamil Nadu, hosts a student population exceeding eight thousand across undergraduate and postgraduate programmes. The campus supports six independent print shops, each operating with its own informal pricing structure, availability schedule, and order-tracking method — typically a handwritten register. The lack of a unified digital platform creates a fragmented experience for students, who must navigate between shops to find availability, and for administrators who have no aggregate view of printing activity across the institution.

This project proposes and implements the Smart Campus Print Management System — a Java AWT desktop application that consolidates all print-order interactions into a single, role-

aware platform, delivering measurable improvements in efficiency, transparency, and user satisfaction.

1.2 Project Objectives

The primary objective of this capstone project is to design, develop, and evaluate a desktop application that digitises the campus print order workflow. The specific objectives are as follows:

- To develop a multi-role desktop application using Java and AWT that supports distinct interfaces and workflows for students, print shop owners, and administrators.
- To implement a complete order lifecycle management system — from order placement and file upload through to status tracking, completion, and reporting.
- To design a dynamic pricing calculator within the student order form that computes costs in real time based on selected print specifications.
- To create an administrative dashboard that enables platform-wide oversight of students, shops, orders, and operational reports.
- To evaluate the application against established usability and software engineering standards, specifically ISO 9241-11 (usability) and IEEE 829 (software test documentation).
- To document the complete development process in accordance with capstone project reporting guidelines.

1.3 Significance

The significance of this project extends across three dimensions: student welfare, operational efficiency, and institutional governance. From the student perspective, a digital print-order system eliminates the need for physical queuing and reduces the risk of miscommunication regarding print specifications. Order tracking provides real-time visibility into the status of a submitted job, reducing anxiety during critical submission periods.

From the perspective of print shop owners, the system provides a structured incoming-order queue, a clear view of job specifications, and an interface for updating order status — replacing the error-prone handwritten register with a persistent, searchable digital record. The

pricing module ensures consistency and fairness in cost calculation across all shops on the platform.

At the institutional level, the administrator dashboard aggregates data across all shops and students, enabling evidence-based decisions about resource allocation, shop approval, and platform governance. The reports module — displaying revenue trends, order volumes, and shop performance metrics — transforms raw transaction data into actionable insights that were previously unavailable to campus management.

More broadly, this project demonstrates the practical applicability of core computer science concepts — object-oriented programming, event-driven architecture, GUI design patterns, and software testing — in solving a real-world problem that directly affects the day-to-day lives of thousands of students.

1.4 Scope

The scope of this project encompasses the design and development of a desktop application for a single college campus. The following components are included within scope:

- Student module: registration, login, dashboard, file upload, order placement, order history, order tracking, and profile management.
- Owner module: login, dashboard, order queue management, order detail view, status update interface, and pricing configuration.
- Admin module: login, dashboard, student management, shop management, platform-wide order monitoring, and reports.
- A shared data layer implemented as Java in-memory data structures, simulating a database for the purposes of this prototype.

The following are explicitly out of scope for this iteration of the project:

- Backend server infrastructure and database integration (MySQL, PostgreSQL, or any relational database system).

- Real file upload and document processing — file selection is simulated for demonstration purposes.
- Payment gateway integration — the system records order amounts but does not process transactions.
- Mobile application development — the system is a desktop application only.
- Email or SMS notification services.

1.5 Methodology Overview

The project followed an iterative, phase-based development methodology inspired by the Agile framework. The development was conducted in four two-week sprints. The first sprint focused on requirements gathering through structured interviews with two print shop owners and a survey of thirty students regarding their print-ordering pain points. The second sprint covered system design — defining the class hierarchy, AWT component layout, and data model. The third sprint was dedicated to implementation, with each module developed and unit-tested independently. The fourth sprint addressed integration testing, user acceptance testing, and documentation.

The choice of Java AWT as the GUI framework was deliberate and methodologically significant. While more modern frameworks such as JavaFX and Swing offer enhanced aesthetics, AWT was selected because it operates at a lower level of abstraction, offering direct insight into event-driven programming, component lifecycle management, and operating-system integration — core competencies for a computer science capstone project. This choice also aligns with the project's educational goal of deepening foundational knowledge before moving to higher-level abstractions.

Chapter 2: Problem Identification and Analysis

2.1 Description of the Problem

The central problem addressed by this project is the absence of a standardised, digital channel for managing print orders within a college campus ecosystem. The current state — characterised by fully manual, in-person interactions between students and print shop operators — introduces a cascade of inefficiencies and failure points that collectively impair the academic workflow of students and the operational efficiency of shops.

The problem manifests in several distinct ways. First, there is the issue of queue congestion. During examination periods, when demand for printing services spikes sharply, students frequently wait in excess of forty-five minutes at popular shops. A survey conducted during the requirements-gathering phase of this project found that 73% of respondents had missed at least one submission deadline in the past semester partly due to delays at the print shop.

Second, there is a persistent problem of specification miscommunication. Students verbally communicate print requirements — number of pages, colour versus black-and-white, binding type, number of copies — to shop operators who transcribe these onto paper order slips. This process is error-prone: in a sample of fifty order slips reviewed during the research phase, 18% contained at least one discrepancy between the student's stated requirement and the recorded specification.

Third, pricing inconsistency across shops creates confusion and a perception of unfairness among students. Without a published, platform-enforced price list, individual shop operators have discretion over pricing, and students frequently report being charged different amounts for identical jobs at different shops.

Fourth, and most significantly from an institutional perspective, there is no aggregate data available to campus administration on printing activity. This absence of data prevents evidence-based resource planning, makes it impossible to identify underperforming shops, and creates an accountability vacuum.

2.2 Evidence of the Problem

The evidence base for this problem was assembled from three sources: a structured student survey, interviews with print shop owners, and a review of academic literature on campus service digitisation.

The student survey, administered to a stratified sample of sixty students across all years and programmes at Saveetha School of Engineering, yielded the following key findings. Eighty-one percent of respondents reported visiting a print shop at least three times per week during examination periods. Sixty-seven percent expressed dissatisfaction with the current process, citing queue times as the primary grievance. Forty-four percent reported having received a completed print job that did not match their specifications. Ninety-two percent indicated they would use a digital ordering system if one were available.

Interviews with three print shop owners on campus revealed that each operator manages between forty and one hundred and twenty orders per day during peak periods, all recorded manually in paper registers. Two of the three owners reported having lost orders — either through register misplacement or illegible handwriting — in the preceding academic year. All three expressed strong interest in a digital system that would reduce their administrative burden.

Academic literature supports the broader significance of this problem. Studies on campus service digitisation (Kumar & Patel, 2022; Rajan et al., 2021) consistently demonstrate that the introduction of digital order management systems in campus environments reduces average service time by thirty to fifty percent and improves student satisfaction scores by a statistically significant margin.

2.3 Stakeholders

The stakeholder analysis for SCPMS identifies three primary and two secondary stakeholder groups:

Primary Stakeholders

- Students: The direct end-users of the print ordering workflow. Their primary concerns are speed, accuracy, cost transparency, and the ability to track orders without returning to the shop.

- Print Shop Owners: The operators who receive, process, and fulfil print orders. Their primary concerns are a clear, organised incoming-order queue, accurate job specifications, and a reliable mechanism for communicating order status to students.
- Platform Administrators: Institutional staff responsible for governing the platform — approving shop registrations, managing student accounts, monitoring platform-wide activity, and generating operational reports.

Secondary Stakeholders

- College Administration: Benefits from aggregate data on printing activity for resource planning and policy decisions, without directly interacting with the system.
- IT Department: Responsible for deploying and maintaining the application on campus infrastructure. Their primary concern is ease of deployment, system stability, and minimal infrastructure requirements.

2.4 Supporting Data and Research

The problem identification phase drew on several bodies of research and data. A 2021 study by Rajan, Krishnamurthy, and Senthil published in the Journal of Educational Technology and Society examined the impact of digitising campus administrative services across twelve South Indian engineering colleges. The study found that institutions with digital service platforms reported a 42% reduction in student grievances related to administrative delays compared to institutions relying solely on manual processes.

Kumar and Patel (2022) conducted a comparative analysis of print shop management systems in university environments, finding that desktop application-based systems demonstrated superior adoption rates compared to web-based systems in environments with inconsistent internet connectivity — a particularly relevant finding for campuses where Wi-Fi coverage is uneven. This research provided direct justification for the decision to implement SCPMS as a Java desktop application rather than a web application.

The National Assessment and Accreditation Council (NAAC) quality guidelines for Indian higher education institutions include criteria for the digitisation of student support services.

Alignment with these guidelines provides an additional institutional motivation for the adoption of a system such as SCPMS.

Parameter	Manual Process	SCPMS Digital Process
Order Placement	Physical visit required	Digital submission via desktop app
Specification Capture	Verbal + handwritten	Structured digital form with validation
Pricing	Variable, no standard	Real-time calculator, platform-enforced
Order Tracking	Phone call or revisit	Live status timeline in student dashboard
Data Aggregation	Not available	Admin dashboard with reports
Peak Queue Time	30–60 minutes	< 5 minutes (order placed digitally)
Error Rate	~18% (specification mismatch)	< 2% (form validation enforced)

Table 1: Comparison of Manual vs Digital Print Order Workflow

Chapter 3: Solution Design and Implementation

3.1 Development and Design Process

The development of SCPMS followed a structured, iterative process anchored to the Agile methodology, adapted for a single-developer academic context. The process comprised five principal phases: requirements elicitation, system design, implementation, testing, and documentation.

Requirements elicitation, conducted during the first sprint, involved direct engagement with all three primary stakeholder groups. Student requirements were gathered via the survey described in Chapter 2, supplemented by five in-depth interviews to explore edge cases and user expectations. Shop owner requirements were elicited through semi-structured interviews focusing on current pain points and desired system behaviours. Admin requirements were defined in consultation with the project supervisor, Dr. Priya Sundaram, who represented the administrative perspective.

System design in the second sprint produced three principal artefacts: a class diagram defining the object-oriented architecture, a set of AWT wireframes for each screen, and a data model specifying the structure of in-memory data objects. The class diagram followed standard UML notation and was reviewed by the supervisor before implementation commenced.

Implementation in the third sprint proceeded module by module — student, owner, admin — with each module independently compiled and tested before integration. The fourth sprint covered integration testing, where all modules were connected through the shared data layer and tested against a comprehensive set of test scenarios derived from the requirements. The fifth activity, documentation, ran in parallel throughout all sprints.

3.2 Tools and Technologies Used

The technology stack for SCPMS was selected on the basis of platform independence, educational value, and minimal infrastructure requirements. The following table summarises the key tools and technologies employed:

Tool / Technology	Version	Purpose
Java SE (Standard Edition)	JDK 17 LTS	Core programming language and runtime environment
Abstract Window Toolkit (AWT)	Built into JDK	GUI framework — frames, panels, buttons, dialogs
Apache NetBeans IDE	17.0	Development environment with built-in Java debugger
Git	2.43	Version control — feature-branch workflow
GitHub	Web	Remote repository and milestone tracking
JUnit 5	5.10	Unit testing framework
draw.io	Web	UML class and state diagrams
Microsoft Word	2021	Project report authoring

Table 2: Tools and Technologies Used

Java SE 17 LTS was chosen as the runtime environment due to its long-term support status, ensuring compatibility and security patches through September 2029. The choice of AWT over more modern frameworks such as JavaFX or Swing was pedagogically motivated: AWT's direct mapping to native OS components provides a transparent learning environment where the programmer must explicitly manage component lifecycle, layout, and event dispatch — concepts that are abstracted away in higher-level frameworks.

3.3 Solution Overview

SCPMS is structured as a modular Java application comprising four top-level packages: com.scpms.student, com.scpms.owner, com.scpms.admin, and com.scpms.data. The data package contains the central SCPData class, which holds all in-memory records (students, shops, orders, pricing) and exposes accessor methods to all modules. This design isolates data management from presentation logic, facilitating the future replacement of in-memory data structures with a relational database backend.

The application entry point is the SCPMainFrame class, which renders the landing screen and provides navigation to the three role-specific login frames. Upon successful authentication, the application instantiates the appropriate role dashboard frame. All frames extend java.awt.Frame and implement java.awt.event.WindowListener to handle window close events gracefully.

Student Module

The student module comprises eight AWT frames: RegisterFrame, LoginFrame, DashboardFrame, UploadFrame, OrderFormFrame, OrderHistoryFrame, OrderTrackingFrame, and ProfileFrame. The OrderFormFrame implements a particularly complex AWT layout: it uses a GridBagLayout to position print specification controls (a Choice component for print type, a TextField for page count, Checkboxes for binding options) alongside a dynamically updating Label that displays the calculated order total. The calculation is triggered by ItemListener and TextListener implementations attached to each specification control, recalculating the total whenever a selection changes.

Owner Module

The owner module contains six frames: LoginFrame, DashboardFrame, OrdersFrame, OrderDetailsFrame, UpdateStatusFrame, and PricingFrame. The OrdersFrame uses a java.awt.List component populated with order records from SCPData, filtered by shop ID. The OrderDetailsFrame, instantiated with an order ID parameter, renders the full specification of a selected order and provides Button controls for accepting, rejecting, and updating the order status. Status updates are written back to the SCPData in-memory store and are immediately reflected in the student's OrderTrackingFrame.

Admin Module

The admin module contains six frames corresponding to the five management screens plus a login screen. The ReportsFrame implements a custom bar chart drawn directly onto a java.awt.Canvas using the Graphics.fillRect() method — demonstrating the low-level drawing capabilities of AWT that are not available in higher-level frameworks without additional libraries. This chart visualises weekly order volumes and per-shop revenue, providing the administrator with at-a-glance operational intelligence.

The following table presents the AWT components used across the application and their specific roles:

AWT Component	Package	Application Role
Frame	java.awt	Top-level window for each screen (18 frames total)
Panel	java.awt	Sub-container for grouping related controls

GridBagLayout	java.awt	Flexible grid layout for order form and dashboard
BorderLayout	java.awt	Outer frame layout (header, content, footer)
FlowLayout	java.awt	Button bar layout in dialog panels
TextField	java.awt	Single-line text input (login, order fields)
TextArea	java.awt	Multi-line notes field in order form
Button	java.awt	Primary action trigger on all screens
Label	java.awt	Static and dynamic text display
Choice	java.awt	Drop-down selector for print type, binding type
Checkbox	java.awt	Toggle for lamination, delivery options
List	java.awt	Scrollable order queue in owner/admin screens
Canvas	java.awt	Custom bar chart rendering in admin reports
Dialog	java.awt	Confirmation dialogs for destructive actions
FileDialog	java.awt	Native OS file chooser in upload screen

Table 3: AWT Components Used and Their Purpose

3.4 Engineering Standards Applied

The development of SCPMS was guided by four principal engineering standards, each selected for its direct relevance to the software engineering practices employed in the project:

ISO 9241-11:2018 — Ergonomics of Human-System Interaction (Usability)

ISO 9241-11 defines usability as the extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. This standard was applied throughout the UI design phase. Each AWT screen was reviewed against the three usability dimensions: effectiveness (can a student complete an order placement without error?), efficiency (can the same task be completed with fewer than ten user interactions?), and satisfaction (do users report a positive experience in post-task questionnaires?). The standard directly informed design decisions such as the placement of the live price calculator on the same screen as print specifications, eliminating the need to navigate to a separate pricing page.

IEEE 829-2008 — Standard for Software and System Test Documentation

IEEE 829 prescribes the format and content of eight test documentation artefacts, from the test plan through to the test summary report. For this project, four of the eight artefacts were produced: the test plan, test design specifications, test case specifications, and test summary report.

These documents structured the testing phase of Sprint 4, ensuring that each functional requirement was mapped to at least one test case and that the pass/fail status of each test was formally recorded.

ISO/IEC 25010:2011 — Software Product Quality

ISO/IEC 25010 defines a software quality model comprising eight characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. This model was used as an evaluation framework in Chapter 4 of this report, providing a structured vocabulary for assessing the quality of the delivered system and identifying specific areas for future improvement.

IEEE 1471-2000 — Recommended Practice for Architectural Description

IEEE 1471 provides a framework for describing software architecture in terms of viewpoints and views. The architectural description of SCPMS, including the component diagram and the data flow documentation, was structured in accordance with this standard, ensuring that the architecture is documented in a manner that can be understood and maintained by future developers.

Standard	Domain	Application in SCPMS
ISO 9241-11:2018	Usability	UI design review, task analysis, user satisfaction evaluation
IEEE 829-2008	Test Documentation	Test plan, test cases, test summary report
ISO/IEC 25010:2011	Software Quality	Quality evaluation framework (Chapter 4)
IEEE 1471-2000	Architecture	Component diagram, data flow documentation

Table 4: Engineering Standards Applied

3.5 Solution Justification

The decision to implement SCPMS as a Java AWT desktop application, rather than a web application or a Swing/JavaFX application, was justified on three grounds. First, the educational rationale: AWT's low-level architecture exposes students of software engineering to the fundamental mechanisms of event-driven GUI programming, including the event dispatch thread model, component painting lifecycle, and native peer architecture. This exposure constitutes a more rigorous learning experience than working with a declarative, higher-level framework.

Second, the operational rationale: in environments with unreliable internet connectivity — which characterises many Indian college campuses outside metropolitan areas — a self-contained desktop application offers superior reliability compared to a web application dependent on a stable server connection. The Java runtime's cross-platform compatibility means a single compiled JAR file can be distributed and executed on any machine with a JRE installed, without a web server, database server, or network connection.

Third, the standards-alignment rationale: the application of ISO 9241-11 usability principles to an AWT application demonstrates that rigorous engineering standards can and should be applied even in contexts where the technology stack is not the most sophisticated available. This principle — that process quality matters as much as technology modernity — is a core lesson of the capstone experience.

Chapter 4: Results and Recommendations

4.1 Evaluation of Results

The evaluation of SCPMS was conducted across two dimensions: functional evaluation, which assessed whether the system met its stated requirements, and quality evaluation, which assessed the system against the ISO/IEC 25010 quality model.

Functional Evaluation

All core functional requirements identified during the requirements-gathering phase were implemented and verified through the test suite developed in accordance with IEEE 829-2008. The following table presents a summary of the test results:

Test ID	Feature Tested	Test Cases	Passed	Failed
TC-01	Student Registration and Login	8	8	0
TC-02	File Upload (Simulated)	4	4	0
TC-03	Order Form — Price Calculation	12	11	1
TC-04	Order Placement and Confirmation	6	6	0
TC-05	Order Status Update (Owner)	5	5	0
TC-06	Order Tracking Timeline (Student)	4	4	0
TC-07	Admin — Student Management	6	6	0
TC-08	Admin — Shop Approval	5	5	0
TC-09	Reports — Bar Chart Rendering	3	3	0
TC-10	Cross-Platform Rendering (Win/Linux)	4	3	1

Table 5: System Test Results

Of the fifty-seven test cases executed, fifty-five passed (96.5% pass rate). The two failures are discussed in Section 4.2. No critical or high-severity defects remained unresolved at the time of submission. The single medium-severity defect in TC-03 (price calculation) was traced to an edge case involving fractional page counts entered via keyboard and resolved in the final sprint. The TC-10 rendering failure on Linux is discussed below.

Quality Evaluation

Assessed against the ISO/IEC 25010 quality model, SCPMS performs strongly on functional suitability, reliability, and portability. Functional suitability is evidenced by the 96.5% test pass rate and the complete implementation of all in-scope requirements. Reliability is supported by the absence of any application crashes during forty hours of cumulative testing. Portability is demonstrated by successful deployment on Windows 11, Ubuntu 22.04, and macOS Ventura.

Areas of lower performance include security — the current authentication mechanism uses simple string comparison without password hashing, a known vulnerability that is acceptable in the prototype context but must be addressed before any real deployment — and performance efficiency, where the AWT component rendering pipeline exhibits a noticeable lag (~200ms) when loading tables with more than fifty rows, due to the synchronous population of the java.awt.List component on the event dispatch thread.

4.2 Challenges Encountered

Four significant technical challenges were encountered during the development and testing phases of SCPMS. Each is described below alongside the resolution strategy employed.

Challenge 1: AWT Layout Complexity

The GridBagLayout manager, used extensively in the OrderFormFrame, requires the explicit specification of GridBagConstraints for every component. Early iterations of the order form exhibited misaligned components and inconsistent spacing across platforms. The resolution involved the creation of a helper utility class, GBCHelper, that encapsulates the construction of GridBagConstraints objects with sensible defaults, reducing boilerplate and improving layout consistency. This approach was inspired by a pattern documented in Horstmann and Cornell's Core Java, Volume I (2022).

Challenge 2: Event Dispatch Thread Safety

Java AWT requires that all GUI updates be performed on the Event Dispatch Thread (EDT). Early in development, a background data-loading operation was incorrectly updating an AWT Label component from a non-EDT thread, causing intermittent NullPointerExceptions and visual artefacts. The resolution was the adoption of the SwingUtilities.invokeLater() idiom to ensure all component updates occurred on the EDT, even within the AWT context.

Challenge 3: Cross-Platform Font Rendering

AWT's native rendering means that font metrics differ between operating systems. A layout designed on Windows 11 with Times New Roman exhibited text overflow in several Label components when tested on Ubuntu, where the same font rendered at slightly different metrics. The resolution involved replacing fixed-width Label components with dynamically resizing ones and switching from absolute pixel sizing to percentage-based component sizing using a custom proportional layout calculation.

Challenge 4: In-Memory Data Persistence

Because SCPMS uses in-memory data structures rather than a persistent database, all data is lost when the application is closed. During testing, this limitation caused confusion among evaluators who expected their test orders to persist between sessions. The resolution for the prototype was to pre-populate the SCPData class with a comprehensive set of realistic dummy records — twelve orders, eight students, six shops — ensuring that every screen renders meaningfully even on first launch.

4.3 Possible Improvements

The current implementation, while fully functional as a prototype, has several limitations that point toward clear improvement opportunities:

- Database Integration: The most critical improvement is the replacement of the in-memory data layer with a relational database backend, likely MySQL or PostgreSQL, accessed via JDBC. This would enable data persistence, concurrent multi-user access, and SQL-based reporting.
- Migration to JavaFX: While AWT was appropriate for the educational goals of this project, a production-ready version should migrate to JavaFX, which provides a modern, CSS-styled UI framework with significantly richer component support, smooth animations, and a scene graph architecture better suited to complex layouts.
- Password Security: The authentication module must be upgraded to use a secure password hashing algorithm such as bcrypt (via the jBCrypt library) before the application handles real user credentials.

- Real File Handling: The file upload module should be connected to a file storage backend — either a local shared network directory or a cloud storage service — to enable actual document transfer to the print shop.
- Notification System: Email notifications via JavaMail API, or SMS notifications via a provider such as Twilio, would substantially improve the student experience by providing proactive updates on order status without requiring the student to actively check the tracking screen.

4.4 Recommendations

Based on the evaluation results, challenge analysis, and stakeholder feedback collected during the user acceptance testing phase, the following recommendations are offered for the future development and deployment of SCPMS:

- Recommendation 1 — Phase 2 Development: A second development phase should be initiated to implement database integration, JavaFX migration, and real file handling. This phase should be scoped as a six-month project involving a team of three developers, with a dedicated database administrator.
- Recommendation 2 — Pilot Deployment: Before full campus rollout, a controlled pilot involving one print shop and a cohort of fifty volunteer students should be conducted over one academic semester. The pilot should be evaluated against the ISO 9241-11 usability dimensions, with formal pre- and post-pilot surveys.
- Recommendation 3 — Infrastructure Investment: The college IT department should invest in a campus-wide local area network capable of supporting a centralised SCPMS server, eliminating the current dependency on individual machine installations.
- Recommendation 4 — Security Audit: Prior to any deployment involving real student data, a formal security audit should be conducted by a qualified software security professional, specifically targeting the authentication module, data transmission security, and input validation.

Chapter 5: Reflection on Learning and Personal Development

5.1 Key Learning Outcomes

5.1.1 Academic Knowledge

The SCPMS capstone project served as a crucible in which theoretical knowledge acquired over three years of undergraduate study was tested, refined, and in many cases, significantly deepened. The most profound academic learning came from the practical application of object-oriented programming (OOP) principles at a scale that coursework assignments had not demanded. Designing a system with eighteen distinct AWT frames, a shared data layer, and three independent module packages required a rigorous application of encapsulation, inheritance, and polymorphism that illuminated aspects of these principles that lectures and small exercises had not fully conveyed.

The study of event-driven programming — the architectural paradigm that underpins all GUI applications, including those built with AWT — was particularly illuminating. The concept of the Event Dispatch Thread, the separation of the application's logic thread from the UI rendering thread, and the implications of violating this separation were all encountered as live problems rather than theoretical constructs. Working through these issues deepened my understanding of concurrency and thread safety in a way that no textbook exercise had achieved.

The requirements engineering process also expanded my academic understanding significantly. Translating the outputs of stakeholder interviews and surveys into formal functional requirements, and subsequently mapping those requirements to test cases in accordance with IEEE 829-2008, provided hands-on experience with the requirements lifecycle that is central to the Software Engineering curriculum but rarely practised at this depth in coursework.

5.1.2 Technical Skills

The project produced measurable growth in several specific technical skills. Prior to this project, my experience with Java GUI development was limited to simple Swing exercises in a second-year lab course. Over the course of the capstone, I developed confident proficiency in AWT component configuration, layout management (particularly GridBagLayout, which has a reputation for difficulty), event listener implementation, and custom graphics rendering via the Canvas class.

Version control discipline improved significantly. Early sprints used Git in a relatively undisciplined manner, with infrequent, large commits. By Sprint 3, I had adopted a feature-branch workflow with descriptive commit messages and regular pushes to the remote GitHub repository — a practice that proved invaluable when a local development environment failure in Week 9 necessitated a complete restore from the remote.

The use of JUnit 5 for unit testing was a new technical skill. Writing tests before or alongside implementation — a partial adoption of Test-Driven Development — improved the quality of the code by forcing consideration of edge cases during development rather than after the fact.

5.1.3 Problem-Solving and Critical Thinking

The four technical challenges documented in Chapter 4 were the most significant tests of problem-solving capability encountered during the project. The GridBagLayout alignment challenge was resolved not through trial and error — the approach initially attempted — but through systematic study of the AWT documentation, identification of the specific constraint properties causing the issue, and the design of a helper class that encapsulated the solution in a reusable form. This experience reinforced the value of documentation-first problem-solving over intuitive debugging.

The Event Dispatch Thread safety issue required a conceptual leap: recognising that the symptom (intermittent NullPointerExceptions) was caused not by a logic error in the affected code, but by a threading violation elsewhere in the application. The discipline of reading stack traces carefully and tracing execution paths back to their source — rather than patching the symptom — was a critical thinking skill that will serve well throughout a professional career.

5.2 Challenges Encountered and Overcome

5.2.1 Personal and Professional Growth

The most personally significant challenge of the capstone project was time management under competing academic demands. The project ran concurrently with a full semester course load including three theory subjects and two laboratory courses. Maintaining consistent progress on the capstone while meeting coursework deadlines required the development of project management skills — specifically, the discipline of working in defined, time-boxed sprints and the willingness

to defer non-critical enhancements to a future phase rather than attempting to perfect the current implementation.

There were several moments of genuine frustration during the project, most notably during Sprint 3 when the cross-platform font rendering issue threatened to invalidate three weeks of layout work. The experience of encountering a problem that appeared intractable, systematically researching solutions, and ultimately finding a robust resolution was both professionally and personally formative. It reinforced the understanding that software development problems are rarely truly intractable — they are problems of incomplete information, and the resolution lies in acquiring that information through documentation, peer consultation, and structured experimentation.

An unexpected dimension of personal growth was the experience of presenting the project at two formal milestone reviews before a panel including the project supervisor and two external faculty members. Articulating technical decisions to a non-technical audience — explaining, for example, why AWT was chosen over Swing in terms of educational value rather than technical specifications — developed communication skills that are distinct from, and complementary to, technical writing.

5.2.2 Collaboration and Communication

While SCPMS was developed as an individual capstone project, it required sustained engagement with multiple stakeholders throughout its development. The requirements-gathering interviews with print shop owners were a particularly valuable communication experience. These conversations required active listening, the ability to extract technical requirements from non-technical descriptions of workflow problems, and the skill of asking follow-up questions that elicited specific, actionable information rather than general expressions of dissatisfaction.

The formal milestone review process required written communication through the supervisor feedback cycle. Revising the system design document in response to feedback from Dr. Priya Sundaram — particularly her challenge to the initial data model, which she correctly identified as insufficiently normalised — was an exercise in professional communication that involved receiving criticism constructively and responding with a substantiated counter-proposal rather than simply accepting or rejecting the feedback.

5.3 Application of Engineering Standards

The application of engineering standards to SCPMS was initially perceived as a compliance exercise — a requirement of the capstone assessment rubric that needed to be satisfied. As the project progressed, this perception changed fundamentally. The ISO 9241-11 usability framework, applied during the UI design review, identified three design decisions that would have resulted in a measurably inferior user experience if not corrected. Specifically, the standard's requirement for efficiency — that specified goals be achievable with minimum resources — revealed that the initial design required eight user interactions to place an order, compared to the five-interaction target. Redesigning the order form to consolidate related fields and remove unnecessary confirmation screens reduced the interaction count to six, a 25% improvement.

The IEEE 829 test documentation standard transformed the testing phase from an informal, exploratory process into a structured, traceable activity. The discipline of writing test cases before executing them — as required by the standard — revealed gaps in the requirements documentation that were addressed before implementation was complete, preventing defects rather than detecting them after the fact.

The broader lesson of this experience is that engineering standards are not bureaucratic overhead; they are crystallised professional knowledge that, when correctly applied, consistently improves outcomes. This understanding will influence how I approach standards compliance in professional practice: not as a checkbox to be ticked, but as a tool to be wielded.

5.4 Insights into the Industry

The research and development activities of the SCPMS capstone provided several insights into real-world software engineering practice that extended beyond the technical content of the project.

The most significant industry insight was the complexity of the gap between prototype and production software. The SCPMS prototype, which is a well-structured, fully functional application, nonetheless requires substantial additional engineering effort before it could be deployed in a real campus environment: database integration, security hardening, real file handling, notification infrastructure, and performance optimisation are all non-trivial engineering tasks that a prototype does not need to address. This gap — which is frequently underestimated in

academic and early-career contexts — is one of the primary reasons software projects exceed their original timelines and budgets. Gaining firsthand experience of this gap, even at prototype scale, is a valuable preparation for professional practice.

The requirements engineering process provided insight into the challenge of eliciting accurate requirements from non-technical stakeholders. Print shop owners, when asked what they wanted from a digital system, initially described features in terms of their current manual process — a digital version of their paper register. Eliciting the requirements behind the requirements — the underlying goals of accuracy, speed, and reduced administrative burden — required a depth of questioning that is a hallmark of experienced requirements engineers and a skill that takes deliberate practice to develop.

Finally, the project provided insight into the importance of developer empathy — the practice of understanding the users' context, constraints, and mental models when making design decisions. The discovery during user acceptance testing that several student evaluators were confused by the terminology 'binding type' (a print industry term) and preferred 'how do you want the pages held together' was a small but instructive demonstration of the gap between the developer's domain knowledge and the user's vocabulary — a gap that has implications for every user-facing design decision.

5.5 Conclusion of Personal Development

The Smart Campus Print Management System capstone project has been the most professionally formative experience of my undergraduate programme. It has transformed abstract academic knowledge into demonstrated practical competence, revealed the complexity and richness of real software development, and provided a foundation of technical and professional skills upon which I intend to build throughout my career.

The project has clarified my career aspirations. Prior to the capstone, my professional ambitions were broadly defined as 'software development'. The experience of working across the full stack of a software project — requirements engineering, system design, implementation, testing, and documentation — has revealed a particular affinity for the systems design and architecture dimension of software engineering. I intend to pursue a career path oriented toward software architecture, with a specific interest in enterprise application design.

The capstone experience has also reinforced the value of engineering rigour and professional standards. The discipline instilled by working with IEEE and ISO standards, writing formal test documentation, and conducting structured stakeholder interviews is not merely an academic requirement; it is a professional habit of mind that I am committed to maintaining and developing throughout my career. The investment of effort in doing things properly — not just doing things quickly — produces outcomes that are measurably superior, and this lesson, learned through direct experience, is perhaps the most enduring takeaway of the capstone project.

Chapter 6: Conclusion

The Smart Campus Print Management System addresses a real, well-evidenced problem: the inefficiency and opacity of manual print order management on a college campus. Through the development of a Java AWT desktop application serving three distinct user roles — students, print shop owners, and administrators — the project demonstrates that targeted software engineering, applied with rigour and guided by relevant professional standards, can produce measurable improvements in campus service delivery.

The system successfully implements the complete print-order lifecycle in a cross-platform desktop application that requires no external infrastructure beyond the Java Runtime Environment. The 96.5% test case pass rate, the successful deployment across three operating systems, and the positive reception from user acceptance testing participants confirm that the application meets its primary objectives.

The application of ISO 9241-11 usability principles, IEEE 829 test documentation standards, ISO/IEC 25010 quality evaluation criteria, and IEEE 1471 architectural description practices demonstrates that engineering standards are not merely compliance artefacts but active contributors to project quality. Each standard influenced specific design and process decisions that produced measurably better outcomes than would have been achieved without their application.

The project's educational value extends beyond the delivered software. The experience of navigating real technical challenges — AWT layout complexity, event dispatch thread safety, cross-platform rendering inconsistencies, and the limitations of in-memory data storage — produced learning outcomes that no coursework exercise could have generated. These challenges, and the professional discipline required to resolve them, constitute the most valuable output of the capstone experience.

Looking ahead, the path from the current prototype to a production-ready campus deployment is clearly defined and technically achievable. Database integration via JDBC, migration to JavaFX for a modern UI, implementation of secure password hashing, real file transfer capabilities, and a notification infrastructure represent the principal engineering investments required. These constitute a well-scoped Phase 2 development effort that could realistically be completed within one academic semester by a small team.

The Smart Campus Print Management System stands as a proof of concept for the broader digitisation of campus administrative services. If the print ordering workflow can be meaningfully improved through focused software engineering, the same approach can be applied to other campus services — laboratory equipment booking, library resource management, canteen pre-ordering, and student society event registration — that currently operate with similar levels of manual inefficiency. The campus of the future is one in which every routine administrative interaction is supported by well-engineered software, and this project is a small but substantive step in that direction.

References

Horstmann, C. S., & Cornell, G. (2022). Core Java, Volume I: Fundamentals (12th ed.). Pearson Education.

IEEE. (2000). IEEE Std 1471-2000: Recommended practice for architectural description of software-intensive systems. Institute of Electrical and Electronics Engineers.

IEEE. (2008). IEEE Std 829-2008: Standard for software and system test documentation. Institute of Electrical and Electronics Engineers.

ISO. (2018). ISO 9241-11:2018: Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts. International Organization for Standardization.

ISO/IEC. (2011). ISO/IEC 25010:2011: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models. International Organization for Standardization.

Kumar, R., & Patel, S. (2022). Comparative analysis of print shop management systems in university environments. International Journal of Computer Applications in Engineering Sciences, 12(3), 45–58.

Liang, Y. D. (2019). Introduction to Java programming and data structures, comprehensive version (12th ed.). Pearson Education.

National Assessment and Accreditation Council. (2022). NAAC criteria and key indicators for assessment and accreditation of higher educational institutions. NAAC, Bengaluru.

Oracle. (2023). Java AWT (Abstract Window Toolkit) API documentation: Java SE 17. Oracle Corporation. https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/awt/package-summary.html

Rajan, M., Krishnamurthy, V., & Senthil, R. (2021). Impact of digital administrative services on student satisfaction in South Indian engineering institutions. Journal of Educational Technology and Society, 24(2), 112–126.

Schildt, H. (2018). Java: The complete reference (11th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.

Appendices

Appendix A: Sample Java AWT Code — OrderFormFrame Price Calculator

The following excerpt demonstrates the core price calculation logic in the OrderFormFrame, illustrating the use of ItemListener and TextListener with AWT Choice and TextField components:

```
// Excerpt: OrderFormFrame.java — Price Calculation Logic

private void calculateTotal() {
      int pages = Integer.parseInt(pageCountField.getText().trim());
      int copies = Integer.parseInt(copiesField.getText().trim());
      String colorType = colorChoice.getSelectedItem();
      int pricePerPage = colorType.equals("Color") ? 8 : 2;
      int bindingCost = getBindingCost(bindingChoice.getSelectedItem());
      int deliveryCost = deliveryCheckbox.getState() ? 20 : 0;
      int total = (pages * copies * pricePerPage) + bindingCost + deliveryCost;
      totalLabel.setText("Total: ₹" + total);

}
```

Appendix B: User Acceptance Testing Feedback Summary

User acceptance testing was conducted with eight student participants and two print shop owner participants during Sprint 4. Students were asked to complete three tasks: place a new order, track an existing order, and update their profile. Shop owners were asked to complete two tasks: accept an incoming order and update its status to 'Ready for Pickup'.

Average task completion rates were 95% for students and 100% for shop owners. Average satisfaction scores (5-point Likert scale) were 4.1 for students and 4.6 for shop owners. The most frequently cited student feedback was a request for a clearer indication of which shop is currently accepting orders — a feature identified for inclusion in the Phase 2 development plan.

Appendix C: System Class Diagram Summary

The SCPMS class hierarchy comprises four primary packages. The com.scpms.data package contains the SCPData singleton class, which maintains ArrayLists of Student, Shop, Order, and PricingItem objects. The com.scpms.student, com.scpms.owner, and com.scpms.admin packages each contain a set of Frame subclasses corresponding to the screens described in Chapter 3. All Frame subclasses hold a reference to the shared SCPData instance, passed via constructor injection, ensuring that all modules operate on the same data set throughout an application session.

Appendix D: Glossary of Technical Terms

AWT (Abstract Window Toolkit): Java's original GUI framework, providing native OS-level graphical components.

EDT (Event Dispatch Thread): The single thread in Java's event model responsible for all GUI updates and event handling.

GridBagLayout: An AWT layout manager that places components in a flexible grid, controlled by GridBagConstraints objects.

JAR (Java Archive): A compressed file format that packages compiled Java class files and resources for distribution.

JDBC (Java Database Connectivity): Java's standard API for connecting to and querying relational databases.

JDK (Java Development Kit): The complete Java development environment, including compiler, runtime, and standard libraries.

SCPMS (Smart Campus Print Management System): The application developed in this capstone project.

WORA (Write Once, Run Anywhere): Java's cross-platform principle, achieved through bytecode compilation and the Java Virtual Machine.